

PROJETO FINAL EMBARCATECH:
MONITOR REMOTO DE QUALIDADE DO AR

Carlos Matheus de Lima Ferreira

cmatheuslf@alu.ufc.br

Introdução

Desde a pandemia do coronavírus em 2022, começou um movimento ainda maior para os cuidados com doenças respiratórias, não só adquiridas por meio de vírus e bactérias, mas também condições com as quais nascemos ou adquirimos devido hábito e ações cotidianas. Assim, ficou cada vez mais necessário ser feita a coleta de dados e análise dos dados ambientais em relação ao ar.

Este projeto visa desenvolver um sistema de monitoramento da qualidade do ar ambiente para monitoramento e acompanhamento de pacientes detentores de dificuldades respiratórias de forma local e também remota, pois dessa forma poderíamos acompanhar de forma rápida, em tempo real e eficaz a condição dos pacientes tanto em ambiente hospitalar quanto em casa para que possamos entender as condições que o paciente está inserido e atuar na remoção dele daquele local se chegarmos a condições críticas que prejudicariam seu bem-estar.

Objetivos

- Coletar dados de qualidade do ar (CO₂, temperatura e umidade) utilizando um sensor SCD30.
- Exibir os dados coletados em um display OLED SSD1306.
- Enviar os dados para um servidor MQTT para monitoramento remoto.
- Implementar alertas quando os níveis de CO₂, temperatura ou umidade ultrapassarem determinados limites.

Justificativa

A crescente preocupação com a qualidade do ar e seus impactos na saúde humana, especialmente para pessoas com doenças respiratórias, justifica o desenvolvimento deste projeto. Com o avanço da tecnologia e a necessidade de monitoramento contínuo, torna-se essencial a implementação de um sistema capaz de fornecer dados ambientais em tempo real.

O sistema "Air Monitor" permitirá o acompanhamento da qualidade do ar em ambientes internos e externos, auxiliando no controle de fatores que podem comprometer a saúde de indivíduos com dificuldades respiratórias. A coleta e análise de dados ambientais possibilitam ações preventivas e corretivas, como a remoção de pacientes de locais com condições críticas que prejudiquem seu bem-estar.

Para viabilizar esse monitoramento, serão utilizados sensores ambientais para medir parâmetros como temperatura, umidade e qualidade do ar. O microcontrolador RP2040, da Raspberry Pi, será empregado devido à sua versatilidade e eficiência para aplicações embarcadas. Essa combinação permite um sistema acessível, escalável e com potencial de integração à Internet das Coisas (IoT), garantindo um acompanhamento remoto eficaz e auxiliando na tomada de decisões em tempo real.

Dessa forma, este projeto contribui significativamente para o bem-estar e qualidade de vida de pacientes com doenças respiratórias, proporcionando uma solução tecnológica inovadora para o monitoramento ambiental.

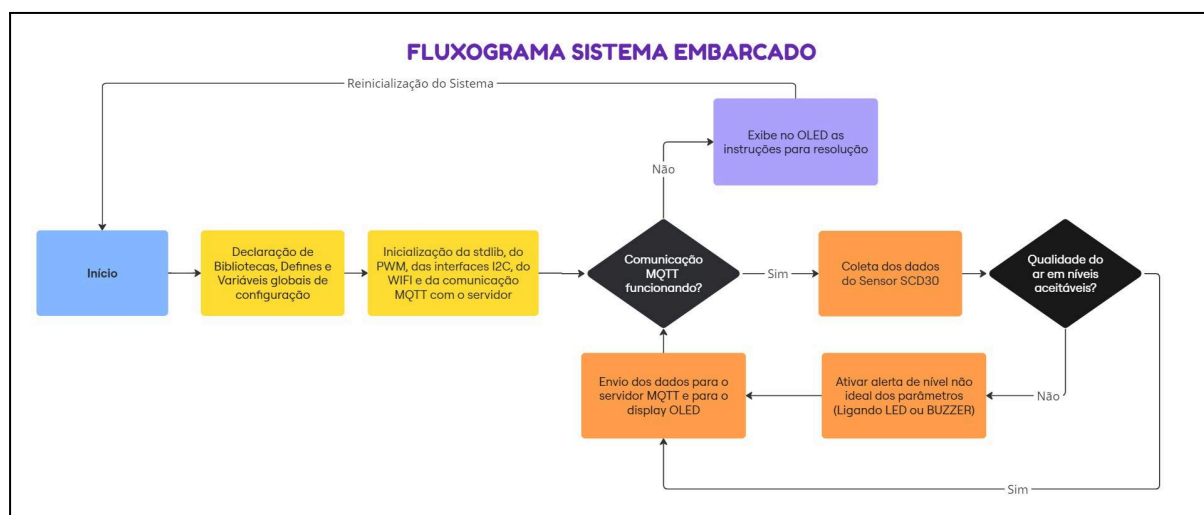
Descrição do funcionamento

1. O microcontrolador Raspberry Pi Pico inicializa os componentes: interface I2C para comunicação com o sensor SCD30 e o display OLED, Wi-Fi para conexão à rede e MQTT para comunicação com o servidor.
2. O sensor SCD30 é configurado para leitura contínua de dados.
3. Em um loop infinito, o microcontrolador lê os dados do sensor, exibe-os no display OLED e envia-os para o servidor MQTT.
4. O microcontrolador verifica se os valores de CO₂, temperatura e umidade estão dentro dos parâmetros aceitáveis. Caso contrário, um LED é acionado como um alerta visual.

5. Um sistema WEB comunica e consome os dados dos tópicos alimentados pelo microcontrolador e então os exibe de forma numérica e através de gráficos para um melhor entendimento do funcionamento dos gráficos.

Fluxograma

Figura 1 - Fluxograma do Sistema Embarcado



fonte: Produzido pelo próprio autor.

Etapas do Processo:

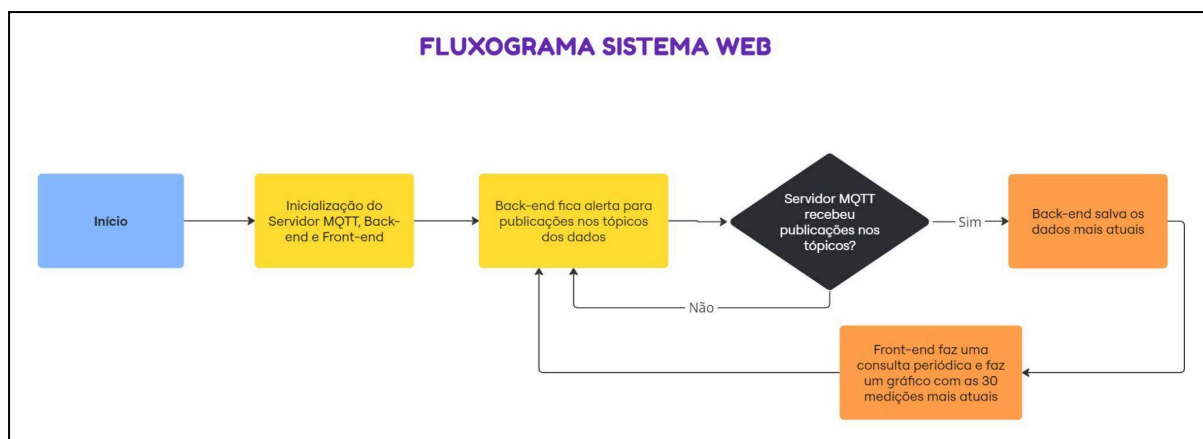
1. **Inicialização:** O sistema inicia, configurando as bibliotecas, interfaces de comunicação (I2C, PWM, Wi-Fi) e a conexão com o servidor MQTT.
2. **Coleta de Dados:** O sistema lê os dados do sensor SCD30, coletando informações sobre CO2, umidade e temperatura.
3. **Verificação da Qualidade do Ar:** Os dados coletados são analisados para verificar se a qualidade do ar está dentro dos limites aceitáveis.
4. **Comunicação com o Servidor MQTT:** Os dados coletados são enviados para o servidor MQTT, permitindo o armazenamento e a visualização dos dados em uma plataforma externa.
5. **Exibição no Display OLED:** O sistema exibe informações relevantes sobre a qualidade do ar no display OLED, como os valores dos sensores e alertas.

6. **Ativação de Alertas:** Caso a qualidade do ar esteja abaixo dos padrões estabelecidos, o sistema aciona um alerta visual ou sonoro (LED ou buzzer).

Componentes:

- **Sensor SCD30:** Responsável por coletar dados sobre a qualidade do ar.
- **Servidor MQTT:** Utilizado para armazenar e compartilhar os dados coletados.
- **Display OLED:** Fornece feedback visual ao usuário.
- **Alerta(LED/Buzzer):** Indica condições de ar insalubre. As quais através de pesquisa foram identificadas como CO2 acima de 1000 PPM, Temperatura acima de 40 °C e Umidade Relativa acima de 80% ou abaixo de 40%.

Figura 2 - Fluxograma do Sistema WEB



fonte: Produzido pelo próprio autor.

Etapas do Processo:

1. **Inicialização:** O sistema inicia com a configuração do servidor MQTT, back-end e front-end.
2. **Subscrição a Tópicos:** O back-end se conecta ao servidor MQTT e se inscreve nos tópicos de interesse, onde os dados serão publicados.
3. **Recebimento de Publicações:** O servidor MQTT envia as publicações para os assinantes, incluindo o back-end.

4. **Armazenamento de Dados:** O back-end armazena os dados recebidos, geralmente em um banco de dados ou em uma estrutura de dados em memória.
5. **Exibição dos Dados:** O front-end realiza consultas periódicas ao back-end para obter os dados mais recentes e os exibe em um gráfico, geralmente com um intervalo de 30 segundos.

Componentes:

- **MQTT:** Protocolo de comunicação leve e eficiente utilizado para publicar-subscriver mensagens.
- **Back-end:** Responsável pela lógica do sistema, incluindo a conexão com o MQTT, o armazenamento de dados e a comunicação com o front-end.
- **Front-end:** Interface do usuário responsável por exibir os dados de forma visualmente atraente.

Código

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/i2c.h"
#include "pico/cyw43_arch.h"
#include "lwip/apps/mqtt.h"
#include "lwip/ip_addr.h"
#include "ssd1306.h"
#include "hardware/pwm.h"

// Credenciais da rede WIFI
char WIFI_SSID[] = "Matheus";
char WIFI_PASSWORD[] = "cartt674586";

// Configs LED PWM
const uint led_r = 13; // GPIO do LED
const uint16_t period_r = 7800; //Período do PWM
const float div_pwm = 16.0; // Divisor do PWM
uint16_t dc_r = 390; // Duty cycle
```

```

// Atribuindo instância cliente mqtt
mqtt_client_t *global_mqtt_client = NULL;

// Define os pinos da interface I2C sc30
#define SCD30_SDA_PIN 0
#define SCD30_SCL_PIN 1
#define SCD30_I2C_PORT i2c0

// Define os pinos da interface I2C OLED
#define OLED_SDA_PIN 14
#define OLED_SCL_PIN 15
#define OLED_I2C_PORT i2c1

// Endereço I2C do SCD30
#define SCD30_ADDRESS 0x61

// Cabeçalhos de funções
void soft_reset_scd30();
void stop_continuous_scd30();
void scd30_read_date(uint8_t *reg, uint8_t *data, uint8_t size);
void mqtt_connection_cb(mqtt_client_t *client, void *arg,
mqtt_connection_status_t status);
void start_mqtt_client(void);

void setup_pwm(uint gpio_pin, uint16_t period, float div, uint16_t
initial_lvl) {
    uint slice;
    gpio_set_function(gpio_pin, GPIO_FUNC_PWM);
    slice = pwm_gpio_to_slice_num(gpio_pin);
    pwm_set_clkdiv(slice, div_pwm);
    pwm_set_wrap(slice, period);
    pwm_set_gpio_level(gpio_pin, initial_lvl);
    pwm_set_enabled(slice, true);
}

// Função de inicialização do SCD30

```

```

void scd30_init(){
    sleep_ms(5000); // Tempo para estabilizar a placa
    printf("Inicializando\n"); // Debug de Inicialização
    soft_reset_scd30(); // Reset do sensor
    sleep_ms(200); // Delay para aplicação do comando
    stop_continuous_scd30();
    sleep_ms(200); // Delay para aplicação do comando
}

// Função de Reset do SCD30
void soft_reset_scd30(){
    uint8_t reg[18]={0}; // Registrador
    uint16_t command = 0xd304; // Código do comando
    uint8_t offset=0; // Posição
    reg[offset++] = (uint8_t)((command & 0xFF00) >> 8); // Primeiros 8
bits
    reg[offset++] = (uint8_t)((command & 0x00FF) >> 0); // Últimos 8
bits
    printf("Soft Reset\n"); // Debug de Reset
    i2c_write_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, &reg, 1, true);
// Envio do comando por I2C
}

void stop_continuous_scd30(){
    uint8_t reg[18]={0}; // Registrador
    uint16_t command = 0x0104; // Código do comando
    uint8_t offset=0; // Posição
    reg[offset++] = (uint8_t)((command & 0xFF00) >> 8); // Primeiros 8
bits
    reg[offset++] = (uint8_t)((command & 0x00FF) >> 0); // Últimos 8
bits
    printf("Stop Continuous scd30\n"); // Debug da Função
    i2c_write_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, &reg, 1, true);
// Envio do comando por I2C
}

// Função para ler os valores do sensor
void get_measurament(float *co2, float *temp, float *rh){
    uint8_t reg[18]={0}; // Registrador

```

```

uint16_t command = 0x0300; // Código do comando
uint8_t offset=0; // Posição
reg[offset++] = (uint8_t)((command & 0xFF00) >> 8); // Primeiros 8
bits
reg[offset++] = (uint8_t)((command & 0x00FF) >> 0); // Últimos 8
bits

uint8_t data[20]={0}; // Buffer para receber dados
scd30_read_date(reg, data, offset); // Chamada da função de leitura
de dados via i2c

// Conversão de bytes para Float

// CO2
unsigned int tempU32 = (data[0] << 24) | (data[1] << 16) | (data[3]
<< 8) | data[4]; // Variável Temporária
*co2 = *(float*)&tempU32;

// TEMPERATURA
tempU32 = (data[6] << 24) | (data[7] << 16) | (data[9] << 8) |
data[10]; // Variável Temporária
*temp = *(float*)&tempU32;

// UMIDADE RELATIVA
tempU32 = (data[12] << 24) | (data[13] << 16) | (data[15] << 8) |
data[16]; // Variável Temporária
*rh = *(float*)&tempU32;

printf("CO2: %.2f\nTemp: %.2f\nRH: %.2f", *co2, *temp, *rh); //
Debug dos valores lidos
}

// Função para ler dados do SCD30
void scd30_read_date(uint8_t *reg, uint8_t *data, uint8_t size) {
    printf("\nreg on read: %02x%02x\n", reg[0], reg[1]); //Debug do
comando dado

```



```

        i2c_write_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, reg, size,
false); // Escrita do comando via I2C
        sleep_ms(300); // Delay para aplicação do comando
        i2c_read_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, data, 18, true);
// Leitura dos dados via I2C
    }

// Funções do Protocolo MQTT

//Função de checagem de conexão
void mqtt_connection_cb(mqtt_client_t *client, void *arg,
mqtt_connection_status_t status) {
    if (status == MQTT_CONNECT_ACCEPTED) {
        printf("Conexão MQTT bem-sucedida!\n"); // Conexão bem sucedida
    } else {
        printf("Falha na conexão MQTT: %d\n", status); // Conexão falha
    }
}

// Inicialização do cliente MQTT
void start_mqtt_client(void) {
    global_mqtt_client = mqtt_client_new(); // Iniciando instância
    if (!global_mqtt_client) {
        printf("Falha ao criar cliente MQTT\n"); // Falha na conexão
        return;
    }

    ip_addr_t broker_ip; // Variavel para receber o IP
    IP4_ADDR(&broker_ip, 192, 168, 1, 105); // Colocando o IP

    struct mqtt_connect_client_info_t client_info = {
        .client_id = "pico_client", // Nick Client MQTT
        .client_user = NULL, // Nick Usuário MQTT
        .client_pass = NULL, // Nick senha MQTT
        .keep_alive = 60, // Tempo de vida
    };
};

```

```

        mqtt_client_connect(global_mqtt_client, &broker_ip, MQTT_PORT,
mqtt_connection_cb, NULL, &client_info); // Iniciando conexão e
configuração
}

// Main
int main() {
    stdio_init_all(); // Iniciando STDlib
    sleep_ms(5000); // Delay de inicialização

    //Configuração do PWM do LED
    setup_pwm(led_r, period_r, div_pwm, 0);

    // Inicializa a comunicação I2C
    i2c_init(SCD30_I2C_PORT, 100 * 4000); // Iniciando a interface I2C
na frequência recomendada pelo fabricante do sensor (40kHz)
    gpio_set_function(SCD30_SDA_PIN, GPIO_FUNC_I2C); // Ativando pino
SDA
    gpio_set_function(SCD30_SCL_PIN, GPIO_FUNC_I2C); // Ativando pino
SCL

    gpio_pull_up(SCD30_SDA_PIN); // Ativando PULL UP pino SDA
    gpio_pull_up(SCD30_SCL_PIN); // Ativando PULL DOWN pino SCL

    i2c_init(OLED_I2C_PORT, 100 * 4000);
    gpio_set_function(OLED_SDA_PIN, GPIO_FUNC_I2C); // Ativando pino
SDA
    gpio_set_function(OLED_SCL_PIN, GPIO_FUNC_I2C); // Ativando pino
SCL

    gpio_pull_up(14); // Ativando PULL UP pino SDA
    gpio_pull_up(15); // Ativando PULL DOWN pino SCL

    ssd1306_t disp; // Instância do OLED

    disp.external_vcc = false;
    ssd1306_init(&disp, 128, 64, 0x3C, i2c1);

```

```

ssd1306_clear(&disp);

//printf("SSID: %s, PASSWORD: %s\n", WIFI_SSID, WIFI_PASSWORD); //
Debug da senha e ssid da rede WIFI

// Iniciando biblioteca para conexão wifi
if (cyw43_arch_init()) {
    printf("Falha ao iniciar\n");
    ssd1306_draw_string(&disp, 0, 0, 1, "Falha ao iniciar");
    ssd1306_show(&disp);
    sleep_ms(2000);
    return 1;
}
cyw43_arch_enable_sta_mode(); //Habilitando modo STA

printf("Conectando ao WIFI...\n"); // Debug de conexão
ssd1306_clear(&disp);
ssd1306_draw_string(&disp, 0, 0, 1, "Conectando ao WIFI...");
ssd1306_show(&disp);

// Conectando ao WIFI
while (cyw43_arch_wifi_connect_timeout_ms(WIFI_SSID, WIFI_PASSWORD,
CYW43_AUTH_WPA2_AES_PSK, 30000)) {
    printf("Falha ao conectar.\n"); // Falha ao conectar
    ssd1306_draw_string(&disp, 0, 9, 1, "Desconectado do WIFI");
    ssd1306_show(&disp);
}

    ssd1306_draw_string(&disp, 0, 9, 1,
"#####");
    ssd1306_show(&disp);
    printf("WIFI Conectado.\n"); // Conexão bem sucedida
    ssd1306_draw_string(&disp, 0, 18, 1, "WIFI Conectado");
    ssd1306_show(&disp);
    ssd1306_draw_string(&disp, 0, 27, 1, "Iniciando...");
    ssd1306_show(&disp);

scd30_init(); // Iniciando sensor SCD30
start_mqtt_client(); // Iniciando MQTT

```

```

// Tópicos para envio das saídas do SCD30
const char *topic_co2 = "air-monitor/co2";
const char *topic_temp = "air-monitor/temp";
const char *topic_rh = "air-monitor/rh";

// Buffer para encapsulamento do payload
char payload[16] = "";

// Variável de armazenamento do sensor SCD30
float co2, temp, rh;

// Variável de erro de comunicação MQTT
err_t err;
sleep_ms(3000);

// LOOP
ssd1306_clear(&disp);
while (true) {

    tight_loop_contents(); // Loop principal

    get_measurement(&co2, &temp, &rh); // Lendo os dados do SCD30
    printf("\nValores lidos - CO2: %.2f Temperatura: %.2f RH: %.2f\n", co2, temp, rh); // Debug de leitura
    // Instruções em caso de erro do MQTT
    ssd1306_draw_string(&disp, 0, 0, 1, "Servidor MQTT OFF");
    ssd1306_draw_string(&disp, 0, 10, 1, "1. Ligue o servidor");
    ssd1306_draw_string(&disp, 0, 20, 1, "2. Reinicie a placa");
    ssd1306_draw_string(&disp, 30, 30, 1, "^^");
    ssd1306_show(&disp);
    ssd1306_clear(&disp);

    // Alerta de níveis por LED/BUZZER
    if(co2> 1000.0 || temp > 40.0 || rh > 70.0 || rh < 40){
        pwm_set_gpio_level(led_r, dc_r);
    }else{
        pwm_set_gpio_level(led_r, 0);
    }
}

```

```

    }

    // Envio para o Servidor MQTT
    if(global_mqtt_client &&
mqtt_client_is_connected(global_mqtt_client)) {

        //Enviando CO2
        sprintf(payload, "%.2f", co2); // Conversão para char*
        err = mqtt_publish(global_mqtt_client, topic_co2, payload,
strlen(payload), 1, 0, NULL, NULL); // Envio do dado
        printf("Mensagem enviada: %s\n", payload); // Debug de dado
enviado

        // Printando no OLED
        ssd1306_draw_string(&disp, 0, 0, 1, "CO2 (PPM): ");
        ssd1306_draw_string(&disp, 55, 0, 1, payload);
        ssd1306_show(&disp);

        //Enviando Temperatura
        sprintf(payload, "%.2f", temp); // Conversão para char*
        err = mqtt_publish(global_mqtt_client, topic_temp, payload,
strlen(payload), 1, 0, NULL, NULL); // Envio do dado
        printf("Mensagem enviada: %s\n", payload); // Debug de dado
enviado

        // Printando no OLED
        ssd1306_draw_string(&disp, 0, 10, 1, "Temp(Co): ");
        ssd1306_draw_string(&disp, 50, 10, 1, payload);
        ssd1306_show(&disp);

        //Enviando RH
        sprintf(payload, "%.2f", rh); // Conversão para char*
        err = mqtt_publish(global_mqtt_client, topic_rh, payload,
strlen(payload), 1, 0, NULL, NULL); // Envio do dado
        printf("Mensagem enviada: %s\n", payload); // Debug de dado
enviado

        // Printando no OLED
        ssd1306_draw_string(&disp, 0, 20, 1, "RH(°): ");
        ssd1306_draw_string(&disp, 40, 20, 1, payload);
        ssd1306_show(&disp);
    }

```

```

        sleep_ms(5000); // Intervalo de leitura de 5s
    }

    return 0;
}

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/i2c.h"
#include "pico/cyw43_arch.h"
#include "lwip/apps/mqtt.h"
#include "lwip/ip_addr.h"
#include "ssd1306.h"
#include "hardware/pwm.h"

// Credenciais da rede WIFI
char WIFI_SSID[] = "Matheus";
char WIFI_PASSWORD[] = "cartt674586";

// Configs LED PWM
const uint led_r = 13; // GPIO do LED
const uint16_t period_r = 7800; //Período do PWM
const float div_pwm = 16.0; // Divisor do PWM
uint16_t dc_r = 390; // Duty cycle

// Atribuindo instância cliente mqtt
mqtt_client_t *global_mqtt_client = NULL;

// Define os pinos da interface I2C sc30
#define SCD30_SDA_PIN 0
#define SCD30_SCL_PIN 1
#define SCD30_I2C_PORT i2c0

// Define os pinos da interface I2C OLED
#define OLED_SDA_PIN 14
#define OLED_SCL_PIN 15
#define OLED_I2C_PORT i2c1

```

```

// Endereço I2C do SCD30
#define SCD30_ADDRESS 0x61

// Cabeçalhos de funções
void soft_reset_scd30();
void stop_continuous_scd30();
void scd30_read_date(uint8_t *reg, uint8_t *data, uint8_t size);
void mqtt_connection_cb(mqtt_client_t *client, void *arg,
mqtt_connection_status_t status);
void start_mqtt_client(void);

void setup_pwm(uint gpio_pin, uint16_t period, float div, uint16_t
initial_lvl){
    uint slice;
    gpio_set_function(gpio_pin, GPIO_FUNC_PWM);
    slice = pwm_gpio_to_slice_num(gpio_pin);
    pwm_set_clkdiv(slice, div_pwm);
    pwm_set_wrap(slice, period);
    pwm_set_gpio_level(gpio_pin, initial_lvl);
    pwm_set_enabled(slice, true);
}

// Função de inicialização do SCD30
void scd30_init(){
    sleep_ms(5000); // Tempo para estabilizar a placa
    printf("Inicializando\n"); // Debug de Inicialização
    soft_reset_scd30(); // Reset do sensor
    sleep_ms(200); // Delay para aplicação do comando
    stop_continuous_scd30();
    sleep_ms(200); // Delay para aplicação do comando
}

// Função de Reset do SCD30
void soft_reset_scd30(){
    uint8_t reg[18]={0}; // Registrador
    uint16_t command = 0xd304; // Código do comando
    uint8_t offset=0; // Posição

```

```

    reg[offset++] = (uint8_t)((command & 0xFF00) >> 8); // Primeiros 8
bits
    reg[offset++] = (uint8_t)((command & 0x00FF) >> 0); // Últimos 8
bits

    printf("Soft Reset\n"); // Debug de Reset
    i2c_write_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, &reg, 1, true);
// Envio do comando por I2C
}

void stop_continuous_scd30(){
    uint8_t reg[18]={0}; // Registrador
    uint16_t command = 0x0104; // Código do comando
    uint8_t offset=0; // Posição
    reg[offset++] = (uint8_t)((command & 0xFF00) >> 8); // Primeiros 8
bits
    reg[offset++] = (uint8_t)((command & 0x00FF) >> 0); // Últimos 8
bits
    printf("Stop Continuous scd30\n"); // Debug da Função
    i2c_write_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, &reg, 1, true);
// Envio do comando por I2C
}

// Função para ler os valores do sensor
void get_measurement(float *co2, float *temp, float *rh){
    uint8_t reg[18]={0}; // Registrador
    uint16_t command = 0x0300; // Código do comando
    uint8_t offset=0; // Posição
    reg[offset++] = (uint8_t)((command & 0xFF00) >> 8); // Primeiros 8
bits
    reg[offset++] = (uint8_t)((command & 0x00FF) >> 0); // Últimos 8
bits

    uint8_t data[20]={0}; // Buffer para receber dados
    scd30_read_date(reg, data, offset); // Chamada da função de leitura
de dados via i2c

    // Conversão de bytes para Float

    // CO2

```



```

    unsigned int tempU32 = (data[0] << 24) | (data[1] << 16) | (data[3]
<< 8) | data[4]; // Variável Temporária
    *co2 = *(float*)&tempU32;

    // TEMPERATURA
    tempU32 = (data[6] << 24) | (data[7] << 16) | (data[9] << 8) |
data[10]; // Variável Temporária
    *temp = *(float*)&tempU32;

    // UMIDADE RELATIVA
    tempU32 = (data[12] << 24) | (data[13] << 16) | (data[15] << 8) |
data[16]; // Variável Temporária
    *rh = *(float*)&tempU32;

    printf("CO2: %.2f\nTemp: %.2f\nRH: %.2f", *co2, *temp, *rh); //
Debug dos valores lidos
}

// Função para ler dados do SCD30
void scd30_read_date(uint8_t *reg, uint8_t *data, uint8_t size) {
    printf("\nreg on read: %02x%02x\n", reg[0], reg[1]); //Debug do
comando dado

    i2c_write_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, reg, size,
false); // Escrita do comando via I2C
    sleep_ms(300); // Delay para aplicação do comando
    i2c_read_blocking(SCD30_I2C_PORT, SCD30_ADDRESS, data, 18, true);
// Leitura dos dados via I2C
}

// Funções do Protocolo MQTT

//Função de checagem de conexão
void mqtt_connection_cb(mqtt_client_t *client, void *arg,
mqtt_connection_status_t status) {
    if (status == MQTT_CONNECT_ACCEPTED) {
        printf("Conexão MQTT bem-sucedida!\n"); // Conexão bem sucedida
    } else {

```

```

        printf("Falha na conexão MQTT: %d\n", status); // Conexão falha
    }
}

// Inicialização do cliente MQTT
void start_mqtt_client(void) {
    global_mqtt_client = mqtt_client_new(); // Iniciando instância
    if (!global_mqtt_client) {
        printf("Falha ao criar cliente MQTT\n"); // Falha na conexão
        return;
    }

    ip_addr_t broker_ip; // Variavel para receber o IP
    IP4_ADDR(&broker_ip, 192, 168, 1, 105); // Colocando o IP

    struct mqtt_connect_client_info_t client_info = {
        .client_id = "pico_client", // Nick Client MQTT
        .client_user = NULL, // Nick Usuário MQTT
        .client_pass = NULL, // Nick senha MQTT
        .keep_alive = 60, // Tempo de vida
    };

    mqtt_client_connect(global_mqtt_client, &broker_ip, MQTT_PORT,
mqtt_connection_cb, NULL, &client_info); // Iniciando conexão e
configuração
}

// Main
int main() {
    stdio_init_all(); // Iniciando STDlib
    sleep_ms(5000); // Delay de inicialização

    //Configuração do PWM do LED
    setup_pwm(led_r, period_r, div_pwm, 0);

    // Inicializa a comunicação I2C

```

```

    i2c_init(SCD30_I2C_PORT, 100 * 4000); // Iniciando a interface I2C
na frequência recomendada pelo fabricante do sensor (40kHz)

    gpio_set_function(SCD30_SDA_PIN, GPIO_FUNC_I2C); // Ativando pino
SDA

    gpio_set_function(SCD30_SCL_PIN, GPIO_FUNC_I2C); // Ativando pino
SCL

    gpio_pull_up(SCD30_SDA_PIN); // Ativando PULL UP pino SDA
    gpio_pull_up(SCD30_SCL_PIN); // Ativando PULL DOWN pino SCL


    i2c_init(OLED_I2C_PORT, 100 * 4000);
    gpio_set_function(OLED_SDA_PIN, GPIO_FUNC_I2C); // Ativando pino
SDA

    gpio_set_function(OLED_SCL_PIN, GPIO_FUNC_I2C); // Ativando pino
SCL

    gpio_pull_up(14); // Ativando PULL UP pino SDA
    gpio_pull_up(15); // Ativando PULL DOWN pino SCL


    ssd1306_t disp; // Instância do OLED


    disp.external_vcc = false;
    ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
    ssd1306_clear(&disp);


    //printf("SSID: %s, PASSWORD: %s\n", WIFI_SSID, WIFI_PASSWORD); //
Debug da senha e ssid da rede WIFI


    // Iniciando biblioteca para conexão wifi
    if (cyw43_arch_init()) {
        printf("Falha ao iniciar\n");
        ssd1306_draw_string(&disp, 0, 0, 1, "Falha ao iniciar");
        ssd1306_show(&disp);
        sleep_ms(2000);
        return 1;
    }

    cyw43_arch_enable_sta_mode(); //Habilitando modo STA


    printf("Conectando ao WIFI...\n"); // Debug de conexão

```

```

ssdl306_clear(&disp);
ssdl306_draw_string(&disp, 0, 0, 1, "Conectando ao WIFI...");
ssdl306_show(&disp);

// Conectando ao WIFI
while (cyw43_arch_wifi_connect_timeout_ms(WIFI_SSID, WIFI_PASSWORD,
CYW43_AUTH_WPA2_AES_PSK, 30000)) {
    printf("Falha ao conectar.\n"); // Falha ao conectar
    ssdl306_draw_string(&disp, 0, 9, 1, "Desconectado do WIFI");
    ssdl306_show(&disp);
}

    ssdl306_draw_string(&disp, 0, 9, 1,
"#####");
    ssdl306_show(&disp);
    printf("WIFI Conectado.\n"); // Conexão bem sucedida
    ssdl306_draw_string(&disp, 0, 18, 1, "WIFI Conectado");
    ssdl306_show(&disp);
    ssdl306_draw_string(&disp, 0, 27, 1, "Iniciando...");
    ssdl306_show(&disp);

scd30_init(); // Iniciando sensor SCD30
start_mqtt_client(); // Iniciando MQTT

// Tópicos para envio das saídas do SCD30
const char *topic_co2 = "air-monitor/co2";
const char *topic_temp = "air-monitor/temp";
const char *topic_rh = "air-monitor/rh";

// Buffer para encapsulamento do payload
char payload[16] = "";

// Variável de armazenamento do sensor SCD30
float co2, temp, rh;

// Variável de erro de comunicação MQTT
err_t err;
sleep_ms(3000);

// LOOP

```

```

ssd1306_clear(&disp);
while (true) {

    tight_loop_contents(); // Loop principal

    get_measurement(&co2, &temp, &rh); // Lendo os dados do SCD30
    printf("\nValores lidos - CO2: %.2f Temperatura: %.2f RH:
%.2f\n", co2, temp, rh); // Debug de leitura
    // Instruções em caso de erro do MQTT
    ssd1306_draw_string(&disp, 0, 0, 1, "Servidor MQTT OFF");
    ssd1306_draw_string(&disp, 0, 10, 1, "1. Ligue o servidor");
    ssd1306_draw_string(&disp, 0, 20, 1, "2. Reinicie a placa");
    ssd1306_draw_string(&disp, 30, 30, 1, "^^");
    ssd1306_show(&disp);
    ssd1306_clear(&disp);

    // Alerta de níveis por LED/BUZZER
    if(co2> 1000.0 || temp > 40.0 || rh > 80.0){
        pwm_set_gpio_level(led_r, dc_r);
    }else{
        pwm_set_gpio_level(led_r, 0);
    }

    // Envio para o Servidor MQTT
    if(global_mqtt_client &&
mqtt_client_is_connected(global_mqtt_client)) {

        //Enviando CO2
        sprintf(payload, "%.2f", co2); // Conversão para char*
        err = mqtt_publish(global_mqtt_client, topic_co2, payload,
strlen(payload), 1, 0, NULL, NULL); // Envio do dado
        printf("Mensagem enviada: %s\n", payload); // Debug de dado
        enviado

        // Printando no OLED
        ssd1306_draw_string(&disp, 0, 0, 1, "CO2 (PPM): ");
        ssd1306_draw_string(&disp, 55, 0, 1, payload);
        ssd1306_show(&disp);
    }
}

```

```

        //Enviando Temperatura
        sprintf(payload, "%.2f", temp); // Conversão para char*
        err = mqtt_publish(global_mqtt_client, topic_temp, payload,
strlen(payload), 1, 0, NULL, NULL); // Envio do dado
        printf("Mensagem enviada: %s\n", payload); // Debug de dado
enviado

        // Printando no OLED
        ssd1306_draw_string(&disp, 0, 10, 1, "Temp(Co): ");
        ssd1306_draw_string(&disp, 50, 10, 1, payload);
        ssd1306_show(&disp);

        //Enviando RH
        sprintf(payload, "%.2f", rh); // Conversão para char*
        err = mqtt_publish(global_mqtt_client, topic_rh, payload,
strlen(payload), 1, 0, NULL, NULL); // Envio do dado
        printf("Mensagem enviada: %s\n", payload); // Debug de dado
enviado

        // Printando no OLED
        ssd1306_draw_string(&disp, 0, 20, 1, "RH(%u): ");
        ssd1306_draw_string(&disp, 40, 20, 1, payload);
        ssd1306_show(&disp);
    }

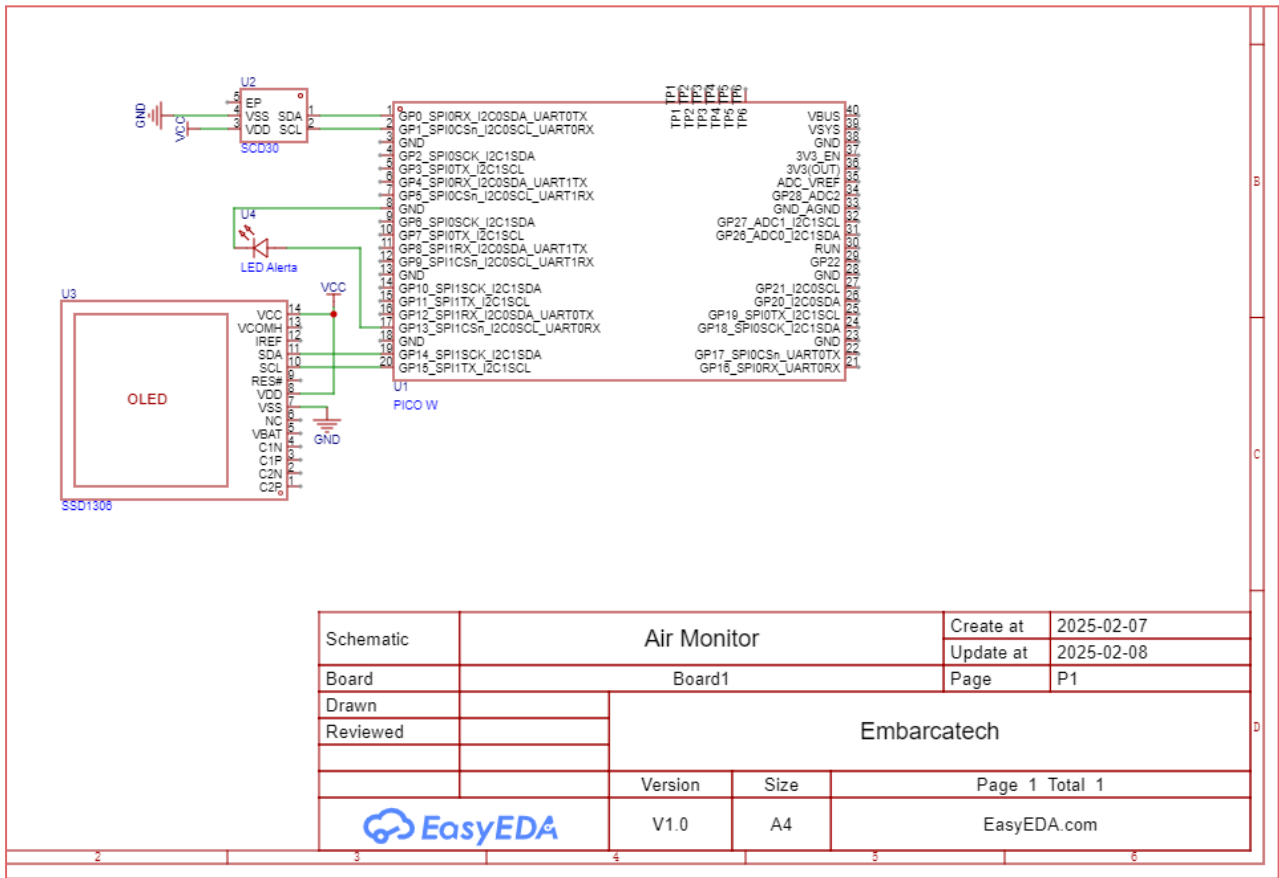
    sleep_ms(5000); // Intervalo de leitura de 5s
}

return 0;
}

```

Esquemático

Figura 3 - Esquemático do Hardware do Sistema Embarcado



fonte: Produzido pelo próprio autor.

Resultados

- Dados são coletados pelo sensor SCD30 a cada 5 segundos.
- São enviados para um servidor MQTT.
- Um sistema WEB recebe e exibe os dados coletados através de gráficos e números.

Acredito que o resultado seja satisfatório visto que os objetivos iniciais foram alcançados, foi possível construir um sistema de monitoramento remoto de condições ambientais que pode ajudar bastante no entendimento e nos cuidados de doenças e síndromes respiratórias, tanto para os médicos que querem ficar de olho no ambiente de um paciente, quanto de uma pessoa que quer monitorar o ambiente ao seu redor para se precaver de situações arriscadas a sua saúde e das pessoas com quem mora.

Referências

Karnati, H. (2023). IoT-Based Air Quality Monitoring System with Machine Learning for Accurate and Real-time Data Analysis. *ResearchGate*. [Online]. Disponível em: https://www.researchgate.net/publication/372074352_IoT-Based_Air_Quality_Monitoring_System_with_Machine_Learning_for_Accurate_and_Real-time_Data_Analysis Acesso em: 05/01/2025.

DE VITO, S. et al. Future Low-Cost Urban Air Quality Monitoring Networks: Insights from the EU's AirHeritage Project. *Atmosphere*, v. 15, n. 11, 1351, 2024. Disponível em: <https://www.mdpi.com/2073-4433/15/11/1351>. Acesso em: 12/01/2025.

GARCÍA, M. R. *et al.* Review of low-cost sensors for indoor air quality: Features and applications. *Applied Spectroscopy Reviews*, v. 58, n. 1-2, p. 1-42, 2023. Disponível em: <https://www.tandfonline.com/doi/full/10.1080/05704928.2022.2085734>. Acesso em: 12/01/2025.

ARAÚJO, T.; SILVA, L.; AGUIAR, A.; MOREIRA, A. Calibration Assessment of Low-Cost Carbon Dioxide Sensors Using the Extremely Randomized Trees Algorithm.¹ *Sensors*, v. 23, n. 13, 6153, 2023. Disponível em: <https://www.mdpi.com/1424-8220/23/13/6153>. Acesso em: 17/01/2025.